

ual:

Coding Three

Creative Robotics

Week 4: 24 October 2025

Today's Agenda

■ Lecture 1: Neural Networks Fundamentals

- What are artificial neural networks
- Neurons, layers, and weights
- How neural networks learn

■ Lecture 2: MNIST & Applications

- Understanding MNIST dataset
- PyTorch implementation

■ Bonus: We'll also explore text detection in computer vision!

■ Lab time:

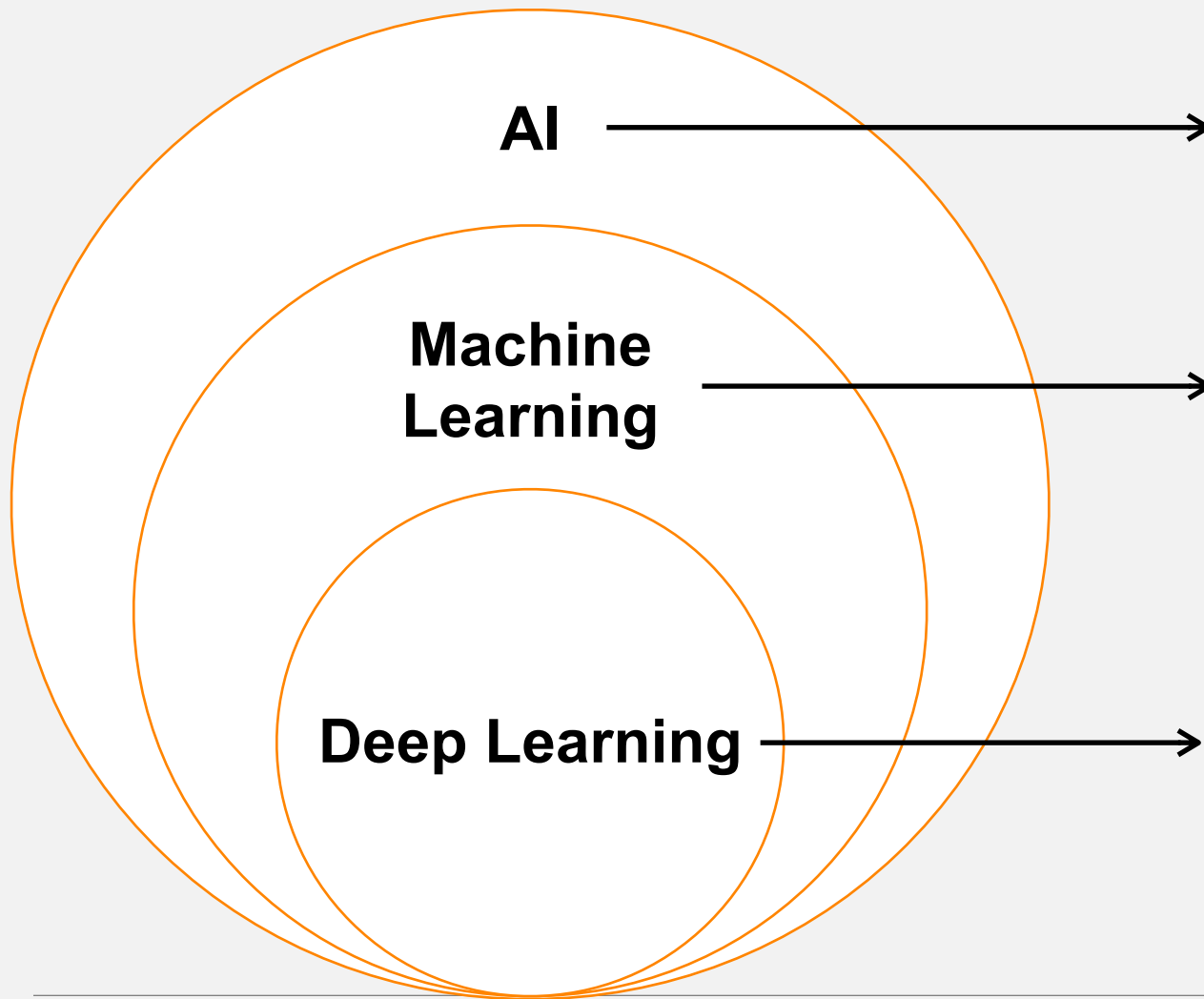
- OpenCV Text Detection and Removal
- PyTorch Setup & Testing
- MNIST demo



What is a Neural Network?



What is Deep Learning?



Artificial Intelligence: Techniques that enable computers to perform tasks typically requiring human intelligence (e.g., perception, reasoning, planning)

Machine Learning: Methods that let systems learn patterns from data **without explicit programming** (supervised learning, unsupervised learning, semi-supervised learning and reinforced learning)

Deep Learning: A subset of ML that learns hierarchical representations from data using **neural networks** (especially deep neural networks)

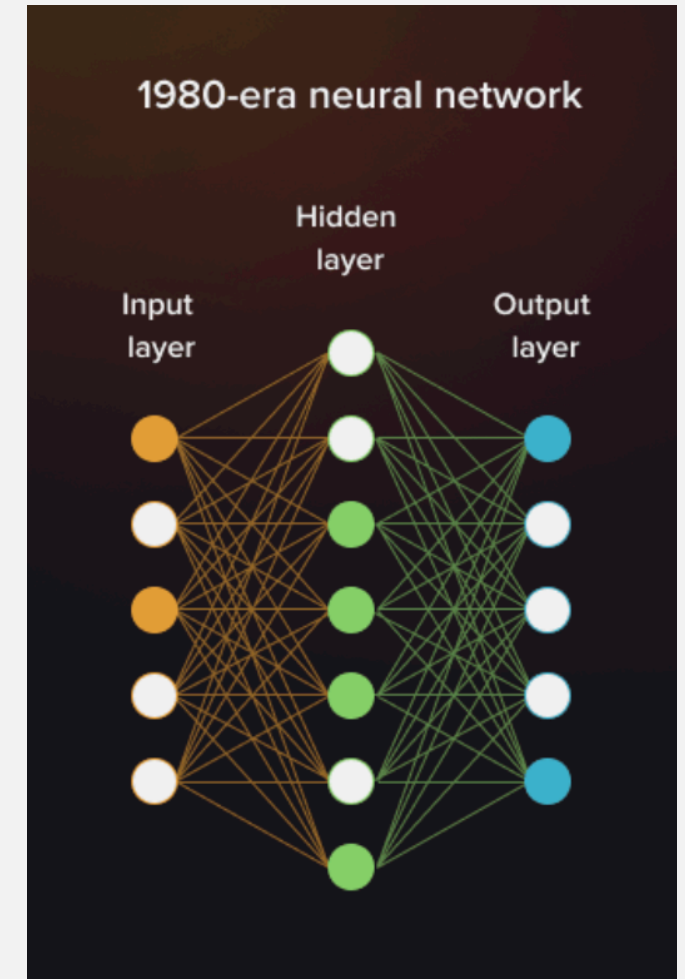
Why Deep Learning?

- **State-of-the-art performance:** Deep learning currently achieves top results on many tasks (image recognition, speech, language, recommendation)
- **Learns features automatically:** Deep networks learn hierarchical features from raw data, reducing the need for manual feature engineering
 - Hand engineered features are time consuming and not scalable in practice
- **Scalability with data and compute:** Performance improves with more data and larger models, making DL suitable for large real-world problems
- **Broad applicability:** Used across domains—vision, NLP, audio, games, healthcare, and recommender systems

What is an Artificial Neural Networks?

- A computational model inspired by our biological brains that maps inputs to outputs
- Built from simple processing units (neurons) arranged in layers: input → hidden → output
- Neurons connect with weighted links; the network learns by adjusting weights and biases from data
- After training, the network can approximate complex functions and make predictions from new inputs
- By 2000s-2010s, deep architecture with many hidden layers became standard for complex tasks (vision, speech, NLP)

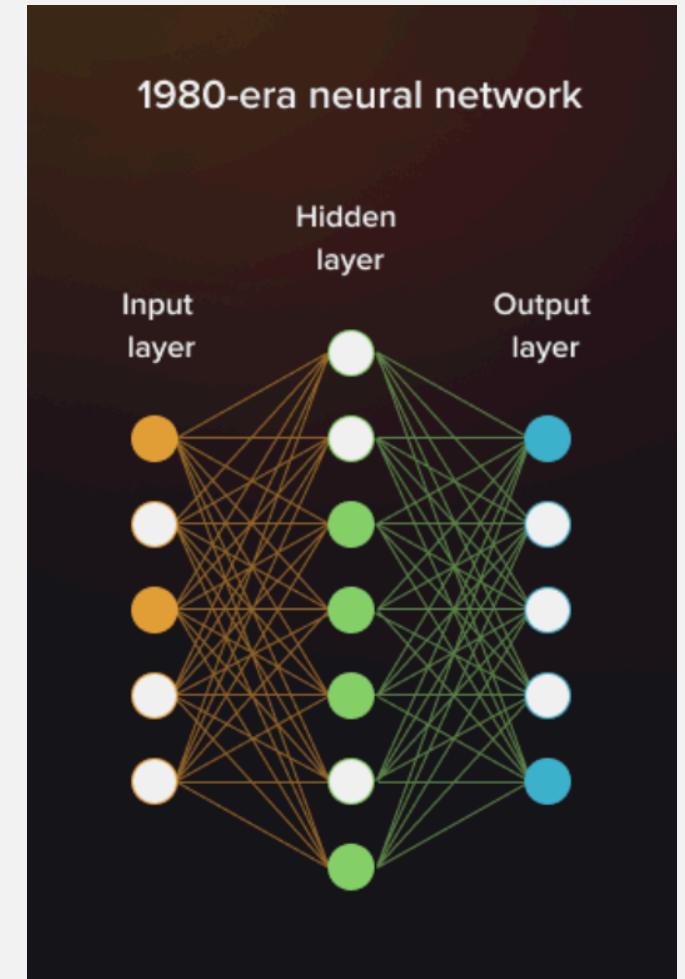
Image source: <https://serokell.io/blog/deep-learning-for-computer-vision>



What is an Artificial Neural Networks?

- Input Layer:
 - Receives raw data
 - One neuron per input feature
 - No computation, just passes data

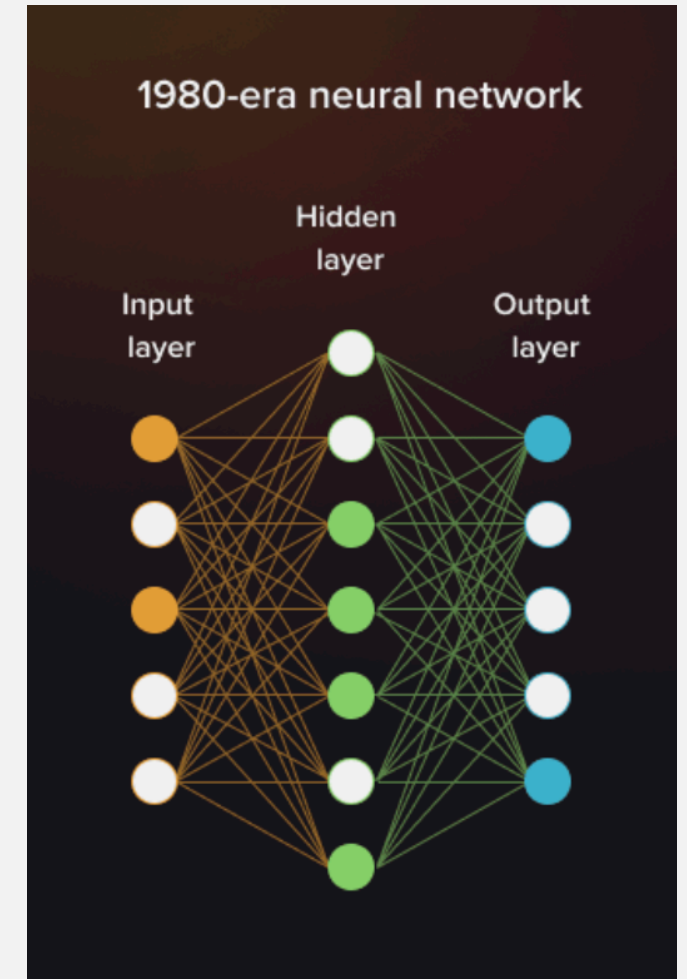
Image source: <https://serokell.io/blog/deep-learning-for-computer-vision>



What is an Artificial Neural Networks?

- Hidden Layer:
 - Process and transform data
 - Extract features and patterns
 - Can have multiple hidden layers (deep learning)
 - Each layer learns different abstractions

Image source: <https://serokell.io/blog/deep-learning-for-computer-vision>

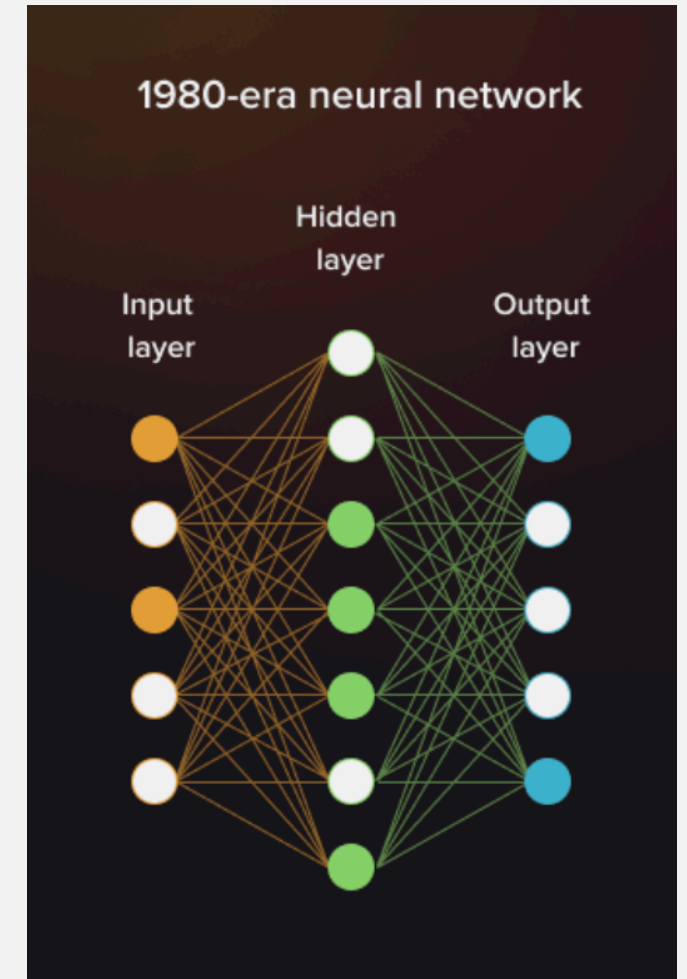


Layers in an Artificial Neural Network

- Output Layer
 - Produces final predictions
 - Number of neurons matches desired outputs
 - For MNIST: 10 neurons (digits 0-9)
 - Often uses **softmax*** for classification

Softmax converts arbitrary scores into probabilities that add up to one. Because it emphasizes larger scores (via exponentiation) while normalizing, it makes the model's confidence comparable across classes and suitable for cross-entropy loss during training

Image source: <https://serokell.io/blog/deep-learning-for-computer-vision>



Our Neurons vs Artificial Neurons

■ Real Neurons in our body:

- Receives signals through dendrites
- Processes information in cell body
- Transmits signals via axon
- Connects to other neurons at synapses

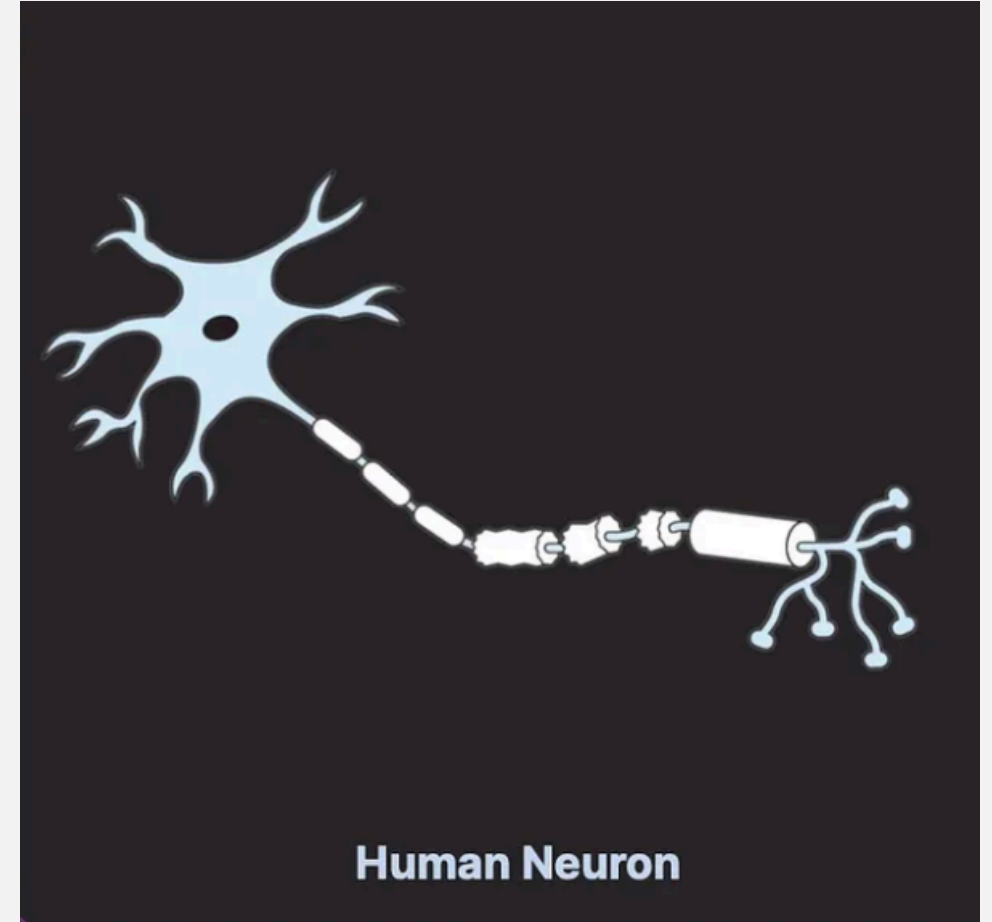


Image source: <https://premvisnnoi.medium.com/understanding-artificial-neurons-the-core-of-deep-learning-0d05a9286c08>

Artificial Neuron (Node)

- Mathematical computation unit
- Receives input values (from data or previous layer)
- Computes weighted sum and adds a bias
- After applying the activation function, we get the neuron's output
- Sends output to neurons to the next layer
- Activation choices (ReLU, sigmoid, etc.) determine nonlinearity and learning behavior
- Many neurons together form a neural network

Image source: <https://premishnoi.medium.com/understanding-artificial-neurons-the-core-of-deep-learning-0d05a9286c08>

Artificial Neuron (Node)

- Simple neuron computation:
$$\text{output} = \text{activation}(\text{weight1} * \text{input1} + \text{weight2} * \text{input2} + \text{bias})$$

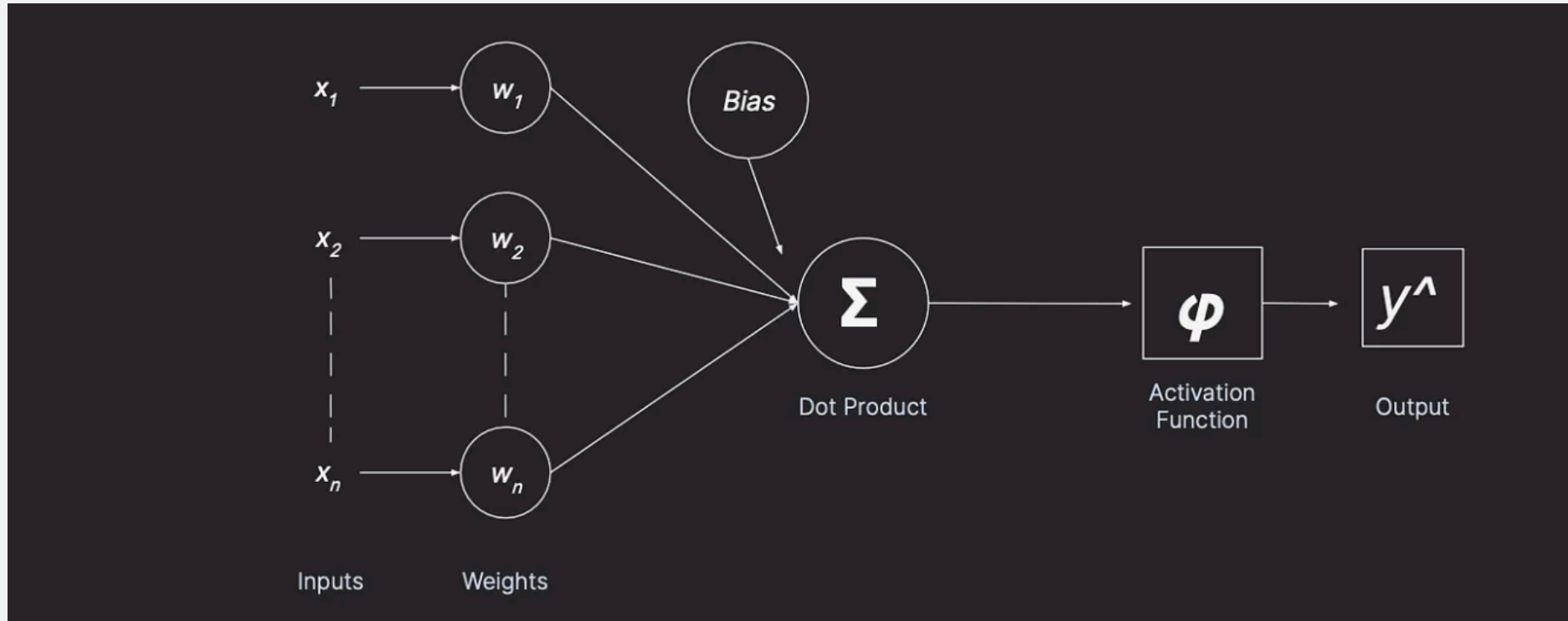


Image source: <https://premishnoi.medium.com/understanding-artificial-neurons-the-core-of-deep-learning-0d05a9286c08>

How Neural Networks Extract Features from Images (Week 6 Preview)

- For this course, we primarily focus on CNNs. It's the most widely used model family for image recognition tasks. But there are others, for example:
 - Recurrent (**RNN**) and Generative Adversarial (**GAN**) serve different roles and are used in specific image-related tasks or in combination of CNNs. They are not primary replacements for CNN in standard image recognition
 - RNN: complements CNNs for sequences (video, captions), not replace CNNs for single-image classifications
 - GANs: in vision, GANs are widely used for image synthesis, image-to-image translation, super-resolution, inpainting, style transfer, domain adaptations
- Next week, we will build a simple model. Week 6, we will fully dive into CNNs

Convolutional Neural Network (CNN)

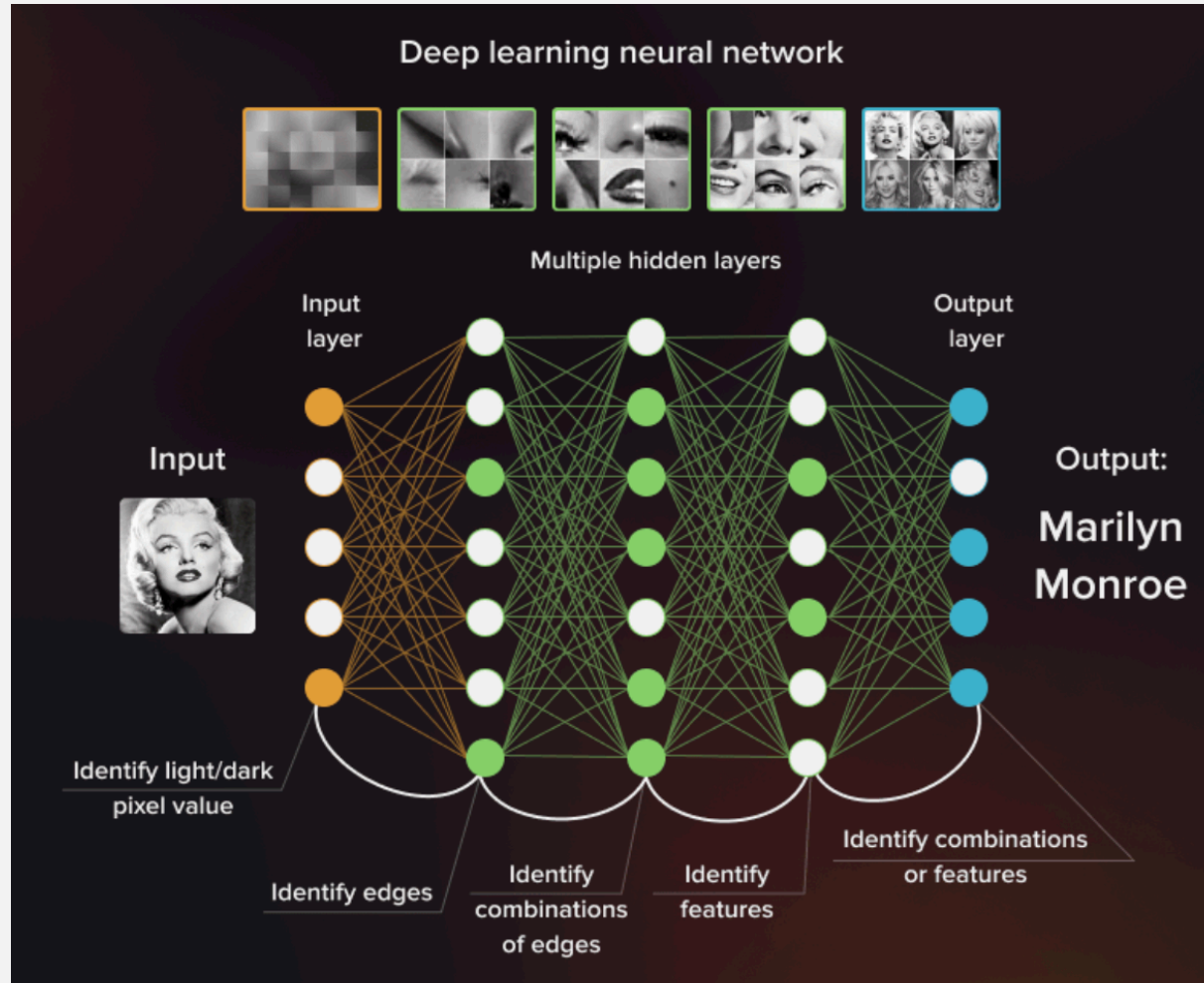
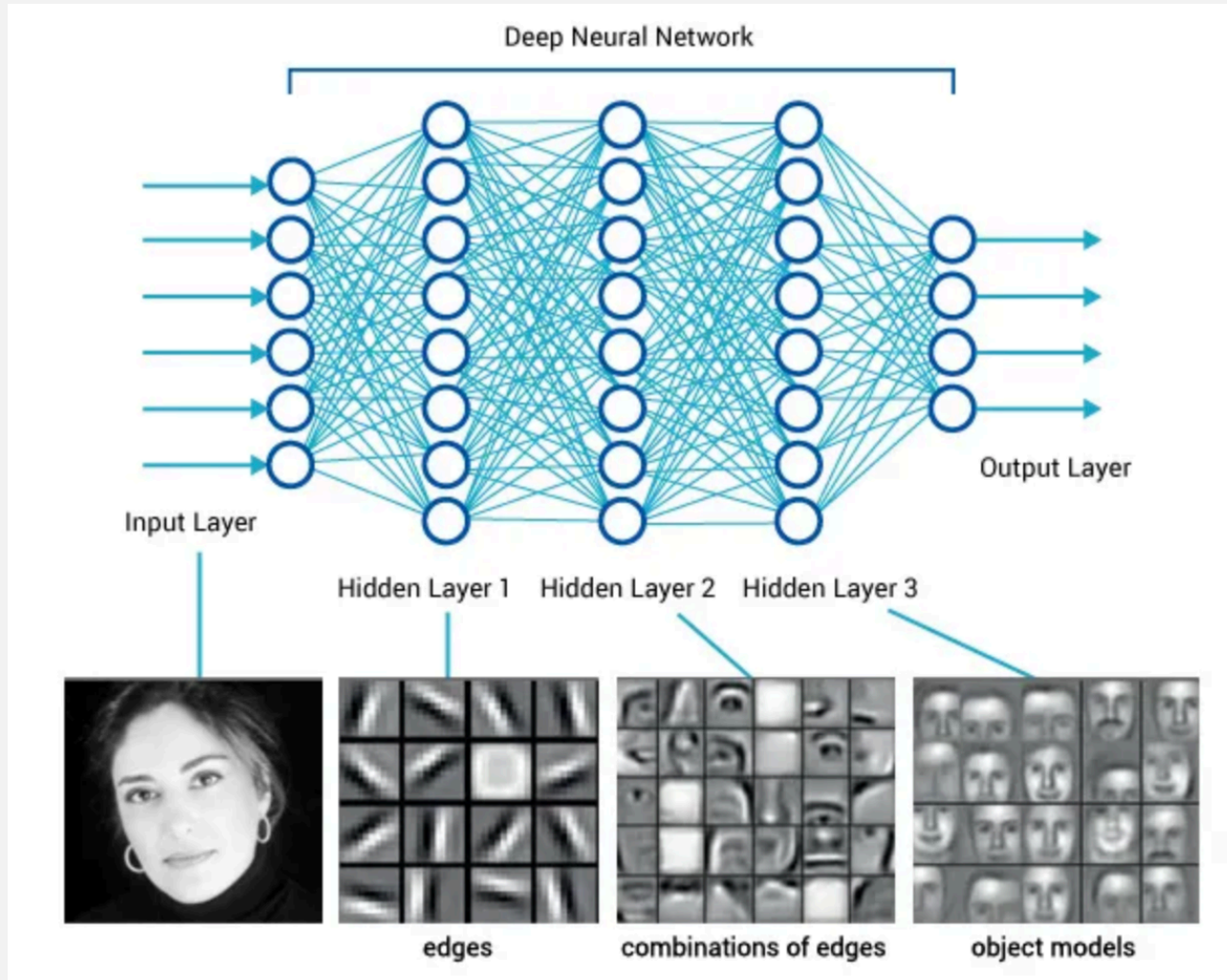


Image recognition:

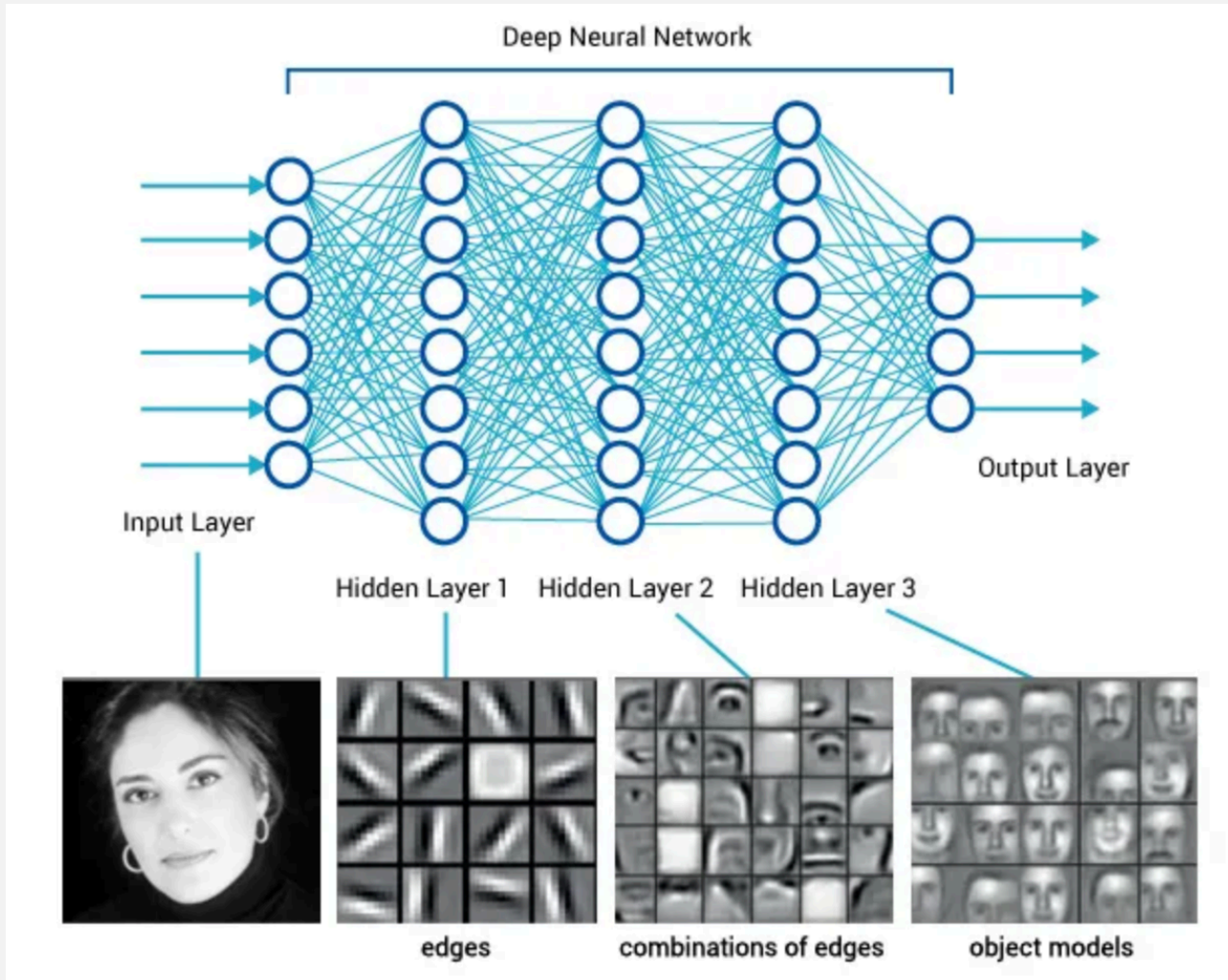
- The network is given an image, which consists of pixels
- In the first convolutional layer, filters are applied to every pixel in the image to create a feature map
- This map is then fed into another layer of filters, which produces another map and so on until the last layer makes its prediction

Convolutional Neural Network (CNN)



- **Low-level features** (what you saw in OpenCV): edges, corners, gradients, simple textures, like Canny edge detector. Here, lines and edges
- **Mid-level features**: combos of edges and textures that form parts: eyes, nose and ears
- **High-level features**: whole objects or categories such as facial structures

Convolutional Neural Network (CNN)



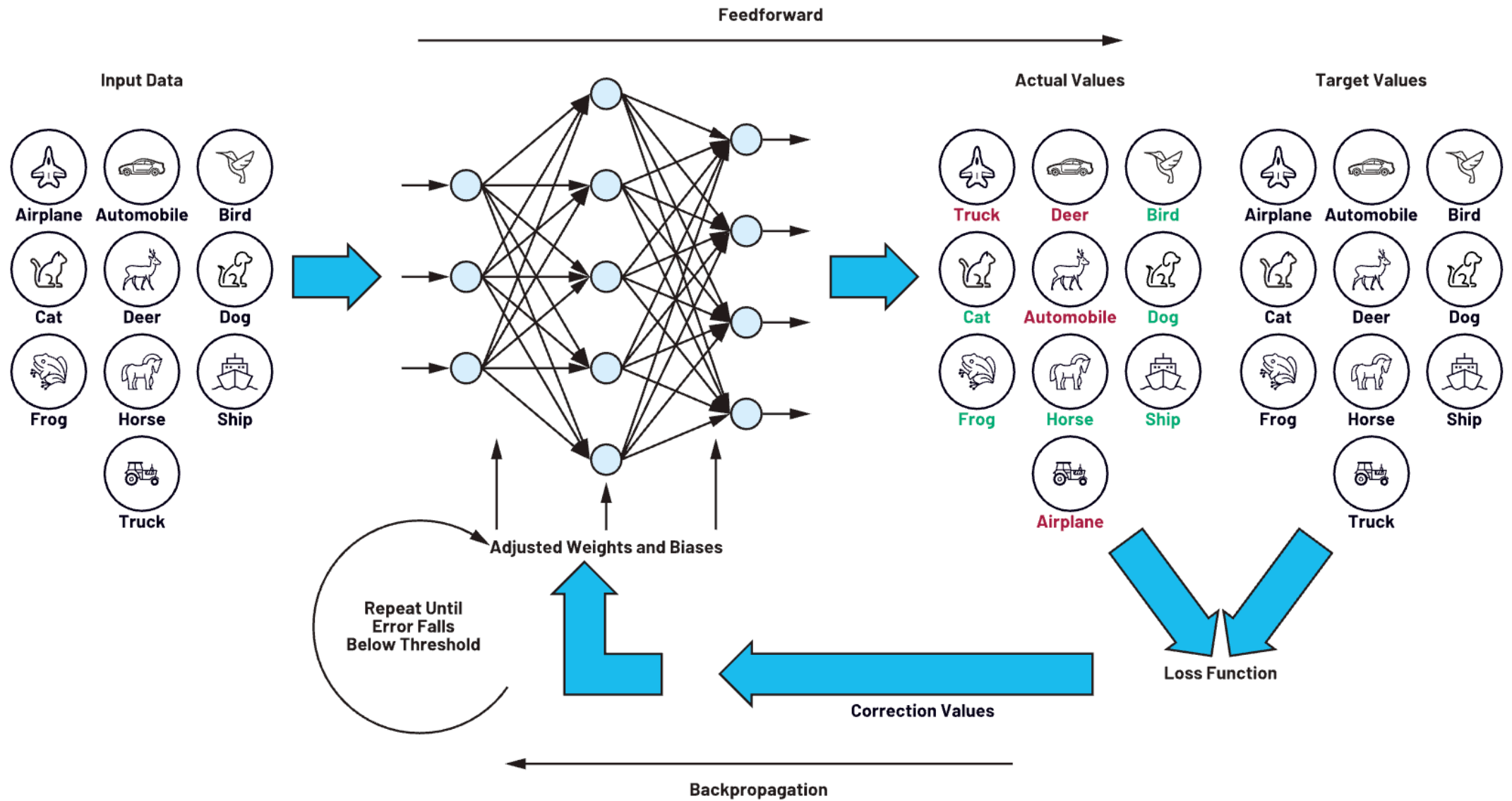
- Early CNN filters often behave like edge detectors you saw in OpenCV in the past few weeks
- Deeper filters compose those edges into parts and objects, but the network discovers and tunes them automatically from data

Training a Neural Network

- **Learning = Optimization**
- Neural networks learn by **minimizing a loss function**
- Key Concepts:
 - **Loss Function:** Measures how wrong predictions are
 - **Gradient Descent:** Method to find minimum loss
 - **Learning Rate:** How big steps to take
 - **Epochs:** Complete passes through data

The Learning Process of Neural Networks

- **Forward Pass:** Model processes input data through its layers to produce a prediction
- **Loss Calculation:** Compare prediction/output with desired result and compute a loss (a number that measures how wrong the prediction is)
- **Backward Pass (back propagation):** Calculate gradients of the loss with respect of each weight
- **Weight Update:** Using an optimizer (for example, Adam, gradient descent) to update the weights to reduce the loss
- **Repeat:** Run many forward/backward passes over the data until the loss is acceptably low (without overfitting) and the model performs well



Overfitting

- You can overfit a model by overtraining
- The model starts to fit too closely with the training data
- A model overfits when it starts to learn this noise



Image source: <https://medium.com/analytics-vidhya/7-ways-to-avoid-overfitting-9ff0e03554d3>

Underfitting vs. Overfitting

- You can underfit a model where it fails to capture the underlying patterns in the data, so it performs poorly on both training and validation sets
- It can be because the model is too small (too few layers/neurons)
- Insufficient training or improperly tuned optimization (too large learning rate, poor initialization)
- How to diagnose:
 - Plot training and validation loss/accuracy across epochs:
 - Underfitting: both curves are high and stay high (no gap, both poor)
 - Overfitting: training loss low, validation loss higher (large gap)

Underfitting vs. Overfitting

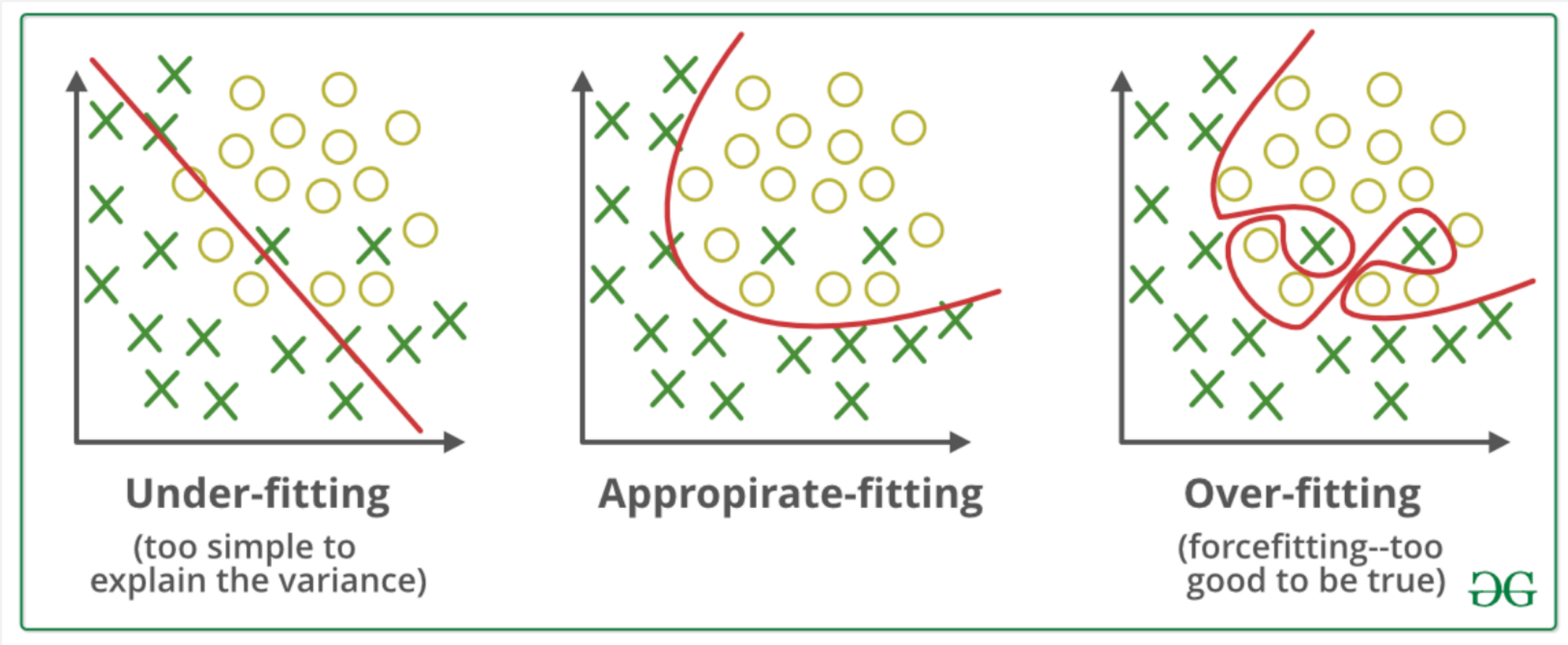
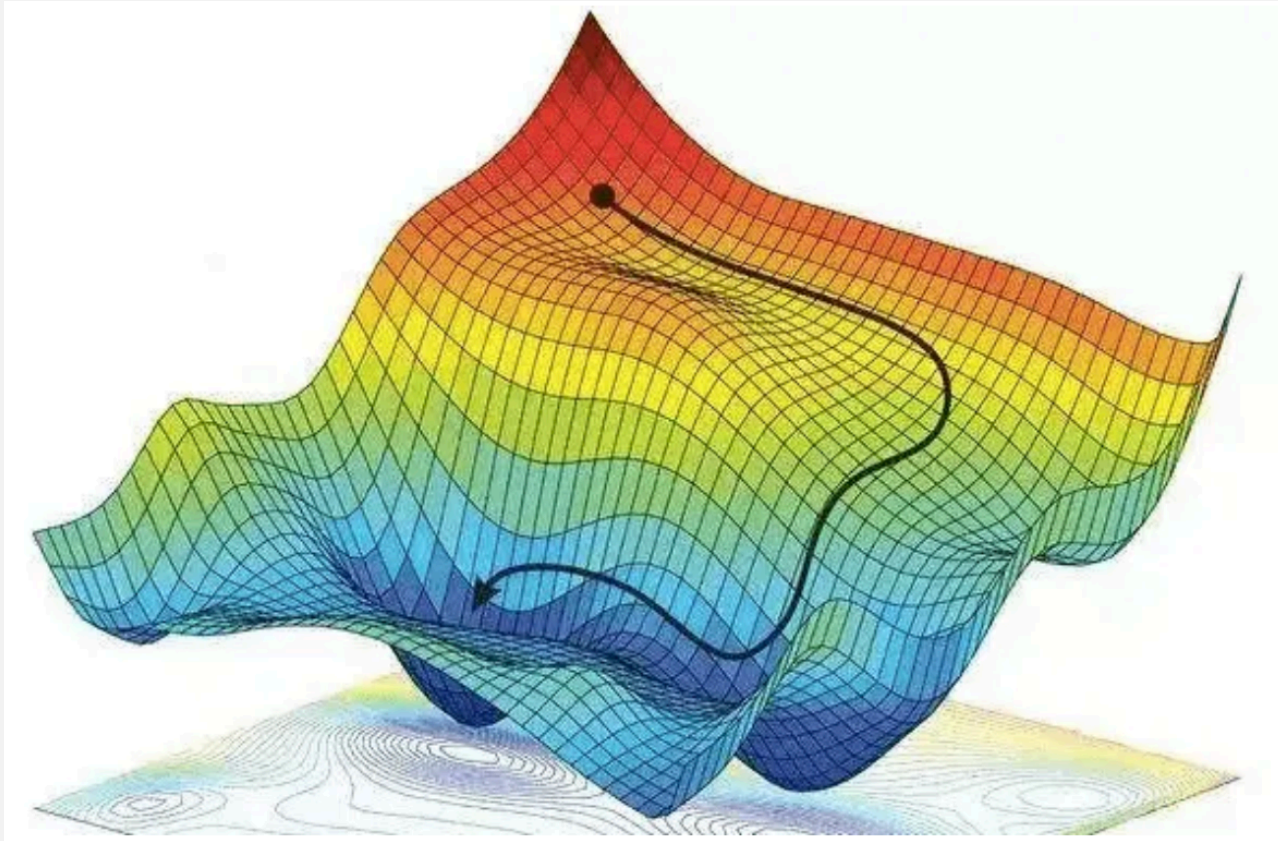


Image source: <https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>

Gradient Descent



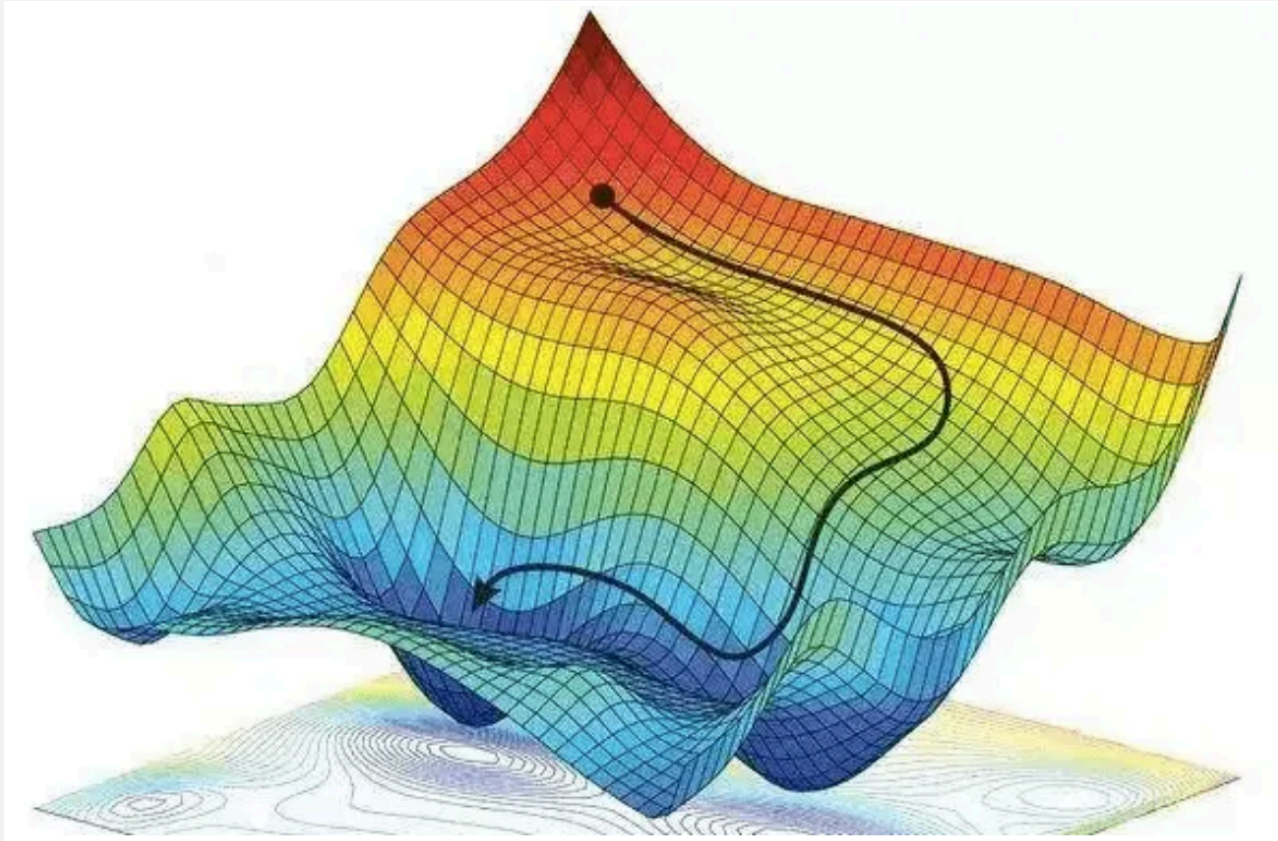
“Minimizing the cost function* is all about **improving the accuracy** of our model. It’s a crucial step in building effective machine-learning models.

Gradient Descent helps us navigate the landscape of our cost function and find the path to the lowest point. It’s like having a guide on our journey toward the most accurate model.”

**Gradient descent minimizes the training objective (the aggregated loss, often called cost).*

Image and quote source: <https://medium.com/@hosamedwee/gradient-descent-1-introduction-333b1134b023>

Gradient Descent



- Analogy: Hiking down a mountain
- Loss = Your altitude
- Gradient = Slope direction
- Learning rate = Step size
- Goal = Reach the valley (minimum)

Image and quote source: <https://medium.com/@hosamedwee/gradient-descent-1-introduction-333b1134b023>

ML x Robotics: Sense-Plan-Act

- Traditional Approach:

- **Sense:**

- **Plan:**

- **Act:**

- Limited to predefined scenarios

ML x Robotics: Sense-Plan-Act

- Traditional Approach:
 - **Sense:** Read sensors directly
 - **Plan:** Rule-based decisions
 - **Act:** Fixed motor commands
- Limited to predefined scenarios

ML x Robotics: Sense-Plan-Act

■ Traditional Approach:

- **Sense:** Read sensors directly
- **Plan:** Rule-based decisions
- **Act:** Fixed motor commands
- Limited to predefined scenarios

■ ML-Enhanced Approach:

- **Sense:** Neural networks process complex inputs
- **Plan:** Learn optimal strategies from data
- **Act:** Adaptive, context-aware actions
- Handles novel situations
- Examples: Self-driving cars, robotic grasping, drone navigation

ML x Robotics: NVIDIA End-to-End Self-Driving

- Remember last week's Edge Detection for Lane Following?

- **Traditional Approach:**

- Canny edge detection finds lane lines

- Rules: If edge angle $>$ threshold, turn

- Explicit modules we can debug

ML x Robotics: NVIDIA End-to-End Self-Driving

■ Remember last week's Edge Detection for Lane Following?

■ Traditional Approach:

- Canny edge detection finds lane lines
- Rules: If edge angle $>$ threshold, turn
- Explicit modules we can debug

■ PilotNet Approach (Neural Networks):

- Single network:
pixels in $\rightarrow$ steering out
- No handcrafted rules or edge detection
- Learns from 500km+ of human driving
- Same goal (lane following),
completely different approach

What Does PilotNet "See"?

- **Neural networks learn their own features** (a measurable property or attribute of the input data that a model uses to make predictions)
- PilotNet's Learned Features using Saliency Maps:
 - Snow-covered road → follows car tracks, ignores missing lines
 - Construction zone → watches equipment edges and curbs
 - Residential street → uses parked cars as guides
- Neural networks handle cases our rules couldn't imagine



Image source: <https://developer.nvidia.com/blog/researching-and-developing-an-autonomous-vehicle-lane-following-system/>

Is This an Example of Supervised Learning?

Is This an Example of Supervised Learning?

■ Yes!

■ What PilotNet does: it learns to map raw front-facing camera images to steering commands (continuous steering angles)

■ Why it's supervised: during training the network is given input–output pairs:

■ Images (inputs) and the corresponding human driver steering angles (labels)

■ The model minimizes a loss (e.g., mean squared error) between its predicted steering and the recorded human steering

■ Data collection: Driving logs record camera frames and synchronized vehicle controls. These paired examples form the supervised training set

■ Outcome: After training, the network predicts steering from new images (inference), approximating the human driver's behavior

Core Foundation Review

Artificial Neuron

- Structural building blocks
- weighted sum + bias

Neural Network

- Stack neurons into layers to form network
- Learnable parameters optimized through back propagation

Training in Practice

- Adaptive optimizers (Adam, gradient descent)
- Mini-batch training
- Overfitting vs. Underfitting

Break Time



MNIST Dataset and Practical Applications



What is MNIST?

- The Modified National Institute of Standards and Technology database
- 70,000 handwritten digit images with an integer between 0 and 9 representing the digit
- 28×28 pixel grayscale images >> 784 pixels
- Simple task (10 classes)
- Already cleaned, and preprocessed data



Image source: <https://datasets.activeloop.ai/docs/ml/datasets/mnist/>

What is MNIST?

- 60,000 training + 10,000 testing samples
- The "Hello World" of machine learning: often used to learn and try ML techniques and pattern recognition methods on real-world data while spending minimal effort on preprocessing and formatting
- Quick training (minutes, not hours)
- Visual feedback (see what the network learns)



Image source: <https://datasets.activeloop.ai/docs/ml/datasets/mnist/>

From MNIST to Real World

■ Skills learned from MNIST can transfer to:

■ Direct Applications:

- Postal code recognition
- Bank check processing
- Form digitization
- License plate reading

■ Extended Applications:

- Medical image analysis
- Facial recognition
- Object detection
- Quality control in manufacturing
- The same principles apply to much more complex vision tasks

Text Detection in Computer Vision

■ Text Detection vs. MNIST

■ MNIST (Single Digits):

- Isolated characters
- Fixed size (28×28)
- Centered and normalized
- Classification only



Image source: <https://datasets.activeloop.ai/docs/ml/datasets/mnist/>

Text Detection in Computer Vision

■ Text Detection vs. MNIST

■ MNIST (Single Digits):

- Isolated characters
- Fixed size (28×28)
- Centered and normalized
- Classification only

■ Text Detection (Real World):

- Multiple characters/words
- Variable sizes and fonts
- Any position/orientation
- Detection + Recognition

Traditional Text Detection (Heuristic-Based)

- Traditional approaches, often leveraging computer vision fundamentals like edge detection, morphology, and region-based analysis
- You the programmer have to tell the computer *exactly* what to look for based on what you know about the text
 - Hand-designed rules features
 - For example, consistent stroke width: There is a key technique called **Stroke Width Transform (SWT)**
 - Color/intensity uniformity: Text is often a single color (like black) on a background (like white)
 - Connected components by strokes: letters are typically made up of small, connected shapes. **Maximally Stable Extremal Regions (MSER)** is designed to find these stable, connected regions

Traditional Text Detection (Heuristic-Based)

- Manual Parameter Settings

- Example: Thresholding

- Grouping: how similar the neighboring components must be grouped as one text block. If the value is set too low, you may split a single word into many parts. Too high, the program may group separate lines of text together

- The system is brittle and easily broken because it relies on your **pre-set rules and parameters**

Text Detection with OpenCV

■ OpenCV: Open Source Computer Vision Library

- A huge toolkit for computer vision
- It has a very fast and powerful tool called **EAST (Efficient and Accurate Scene Text Detector)**
- The **deep learning model** looks at millions of images of text and *learned* what text looks like, no matter the angle or background
- It is trained using **labeled images** where humans have drawn the **precise bounding boxes** (the **correct output/answer**) around every piece of text
- During training, the model's job is to learn the features (what the models "see") of the image that map directly to these human-provided location coordinates and a confidence score
- **Output:** A simple bounding box around the detected text

Lab: Text Detection and Removal using OpenCV

- *This is a good exercise for those of you want to try combining two different actions for your Assignment #1 (part of your assessment)*
- Instead of using just *one* rule like the traditional way, the **pipeline** lets us combine multiple tools in a series of steps.
- The pipeline here for text detection and removal is:
 - Image → Detect Text → Recognize Text → Removal → Output
- OpenCV “glues” the different algorithms to connect them

Lab: Text Detection and Removal using OpenCV

- OpenCV will use the EAST model to detect text
- Then use **Inpainting** to remove the text
- Inpainting basically fills in the missing area with a cleverly blended average of the surrounding colors and textures
- In the lab, you will see both **Navier-Stokes Inpainting** (`cv2.INPAINT_NS`) and **Tele's Inpainting** (`cv2.INPAINT_TELEA`) to remove the text
- For more technical details, you can read this [blog post](#) that today's lab is adapted from

Pipeline Code Structure

- Useful if you want to combine 2+ OpenCV actions for Assignment #1
- Like writing an essay, code also has a structure and hierarchy

Section	Code Task	Purpose	Example
Preamble	<code>import</code> statements and global constants	Load all necessary libraries	<code>import cv2</code>
Setup	Load the image and create copies	read raw data/ pixels and keep an untouched original	<code>img = cv2.imread(. .)</code>
Preprocessing	prepare image for detection	normalize/denoise/resize so algos work reliably	<code>cv2.cvtColor</code>

Pipeline Code Structure

Section	Code Task	Purpose	Example
Action 1	Run first algorithm, for example, edges	Produce an intermediate feature map	<code>cv2.Canny</code>
Intermediate processing	Clean/transform action1 output	Connect/filter/shape regions for the next step	<code>cvs.threshold</code>
Action 2	Run the second algorithm, for example, blob analysis	Detect final object/features using filtered inputs	<code>cv2.SimpleBlobDetector_create</code>
Visualization/ Output	Draw results and export view	Present/verify results and save outputs	<code>cv2.drawContours</code>

Lab Time